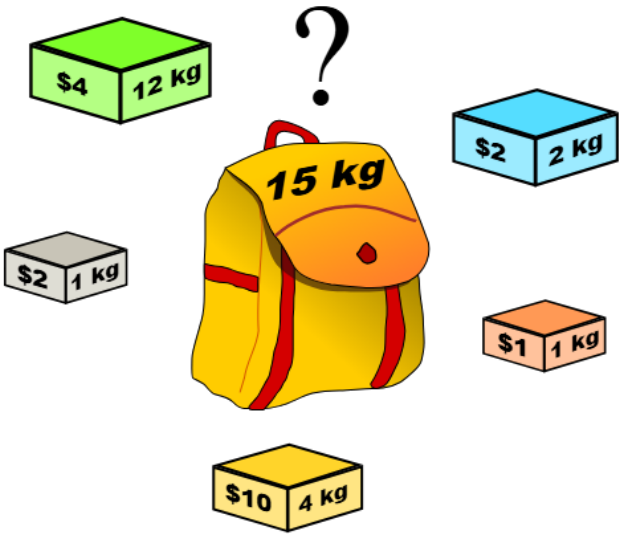
Tugas Besar 1

Penerapan Algoritma *Local Search* untuk Optimasi Pemilihan dan Penataan Paket dalam Peti Kemas

IF3170 Intelegensi Artifisial

Semester 1 Tahun 2026/2027



Disusun oleh:

Three Agents

Muhammad Aufar Rizqi Kusuma	(13524061)
Arina Azka	(13524049)
Michael James Liman	(13524106)

Laboratorium Intelegensi Buatan
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **2**
1.1 *Backstory* . . . 2
1.2 *Overview* Permasalahan . . . 2
2 Landasan Teori **4**
2.1 Algoritma Hill Climbing . . . 4
2.2 Algoritma Simulated Annealing . . . 5
2.3 Genetic Algorithm . . . 6
3 Implementasi Algoritma **8**
3.1 Representasi State . . . 8
3.2 *Constraint* Permasalahan . . . 9
3.3 *Successor Function* . . . 11
3.4 *Objective Function* . . . 11
3.5 Formulasi *Initial State* . . . 11
3.6 Implementasi Hill Climbing . . . 11
3.7 Implementasi Simullated Annealing . . . 11
3.8 Implementasi Genetic Algorithm . . . 11
4 Hasil Eksperimen dan Analisis **12**
4.1 Metodologi Eksperimen . . . 12
4.2 Hasil Eksperimen dan Analisis tiap Algoritma . . . 12
4.2.1 Hill Climbing . . . 12
4.2.2 Simulated Annealing . . . 12
4.2.3 Genetic Algorithm . . . 12
4.3 Analisis Keseluruhan . . . 12
5 Kesimpulan dan Saran **13**
5.1 Kesimpulan . . . 13
5.2 Saran . . . 13

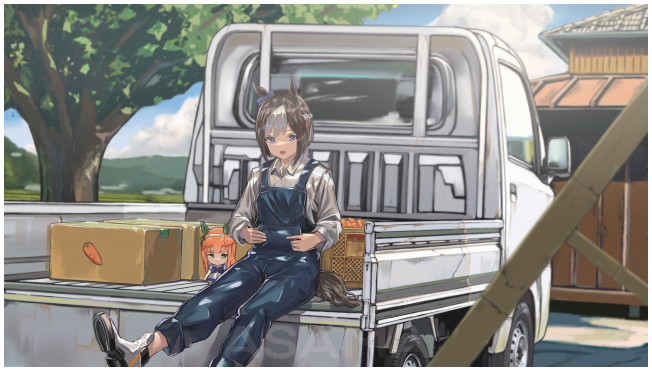
Bab 1

Pendahuluan

1.1 *Backstory*

Keluarga Mejiro sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, Mejiro McQueen menawarkan posisi driver truk uji coba kepada Special Week, karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama Mejiro Express dengan slogan "Lebih kilat dari Umamusume".

Karena Special Week sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.



Gambar 1.1: Special Week, driver truk handal (Sumber: Manatsu no Gin Hourui Uma C108 Doyoubi Higashi Chiku Ho-10a on X)

1.2 *Overview* Permasalahan

Tugas Besar 1 mata kuliah IF3170 Inteligensi Artifisial menugaskan mahasiswa untuk mencari konfigurasi pemuatan paket yang optimal pada sebuah truk peti kemas. Secara umum, paket memiliki ukuran, berat, dan nilai yang beragam, sedangkan ruang serta kapasitas angkut truk terbatas. Tujuan optimasi adalah memilih dan menata paket agar nilai total paket yang berhasil dimuat menjadi sebesar mungkin.

Permasalahan ini tidak hanya berkaitan dengan pemilihan paket bernilai tinggi, tetapi juga dengan kemungkinan penataannya di dalam ruang tiga dimensi. Susunan paket harus tetap layak untuk diangkut dengan memperhatikan keterbatasan ruang, kestabilan tumpukan, kondisi barang pecah-belah, dan kapasitas truk. Beragamnya kombinasi paket dan penataannya membuat pencarian konfigurasi terbaik menjadi sulit dilakukan secara menyeluruh.

Untuk mencari solusi, tugas ini meminta implementasi tiga algoritma pencarian, yaitu satu varian Hill Climbing,

Simulated Annealing, dan Genetic Algorithm. Algoritma-algoritma tersebut mengeksplorasi konfigurasi secara bertahap dari solusi awal yang dibentuk secara acak, lalu berusaha menemukan konfigurasi dengan nilai objektif yang lebih baik. Kinerja pencarian kemudian dibandingkan melalui kualitas solusi, perubahan nilai objektif selama proses, durasi, dan konsistensi hasil eksperimen.

Bab 2

Landasan Teori

2.1 Algoritma Hill Climbing

Hill Climbing adalah algoritma pencarian lokal yang berusaha meningkatkan kualitas satu solusi secara bertahap. Algoritma memulai pencarian dari satu konfigurasi, membentuk sejumlah konfigurasi tetangga, lalu berpindah ke tetangga yang memberikan nilai objektif lebih baik. Pencarian berhenti saat tidak ditemukan tetangga yang meningkatkan nilai tersebut. Varian *steepest-ascent* yang digunakan pada pseudocode memilih tetangga dengan nilai objektif tertinggi dari semua kandidat yang diperiksa [1, 2].

Karakteristik utama Hill Climbing adalah penggunaan satu konfigurasi aktif dan keputusan yang hanya bergantung pada nilai konfigurasi tetangganya. Algoritma ini tidak menyimpan pohon pencarian atau seluruh jalur yang ditempuh. Karena setiap langkah memilih perbaikan lokal, nilai objektif konfigurasi yang dikunjungi meningkat secara monoton pada varian *steepest-ascent* dengan perpindahan yang hanya menerima peningkatan ketat. Dengan ruang solusi berhingga, proses tersebut berakhir pada konfigurasi yang tidak memiliki tetangga lebih baik, yaitu optimum lokal [1].

Keunggulan Hill Climbing terletak pada kesederhanaan implementasi dan kebutuhan memori yang rendah dibandingkan metode yang menyimpan banyak kandidat pencarian. Algoritma ini juga dapat menghasilkan perbaikan dengan cepat apabila fungsi objektif dan pembentukan tetangga dapat dihitung secara efisien. Namun, hasilnya bergantung pada konfigurasi awal dan definisi tetangga. Pencarian dapat berhenti pada optimum lokal yang bukan optimum global, atau kesulitan bergerak pada daerah datar dan punggung sempit pada lanskap objektif. Oleh sebab itu, Hill Climbing tidak menjamin solusi terbaik secara global [1, 2].

Hill Climbing sesuai digunakan ketika ruang solusi besar, evaluasi kualitas solusi relatif murah, dan solusi yang baik sudah berguna meskipun belum terbukti optimal. Metode ini kurang sesuai apabila dibutuhkan jaminan optimum global atau apabila perpindahan lokal jarang menghasilkan arah perbaikan yang informatif. Penggunaan beberapa titik awal atau varian yang mengizinkan langkah menyamping dapat mengurangi sebagian risiko terjebak, tetapi tidak mengubah sifat dasar metode pencarian lokal.

Dalam persoalan pemuatan paket, konfigurasi awal dibentuk secara acak sesuai spesifikasi. Algoritma kemudian membandingkan konfigurasi tetangga yang diperoleh melalui langkah perubahan susunan paket yang diizinkan, lalu memilih konfigurasi layak dengan nilai total paket yang lebih tinggi. Proses berlanjut sampai tidak ada tetangga yang meningkatkan nilai objektif. Pseudocode berikut menunjukkan varian *steepest-ascent* tanpa langkah menyamping atau pengulangan dari titik awal baru.

Algorithm 1: Steepest-Ascent Hill Climbing

Data: Konfigurasi awal s_0, fungsi tetangga N, fungsi objektif f
Result: Konfigurasi optimum lokal s

```
s ← s_0
while N(s) ≠ ∅ do
    s' ← arg max_{u∈N(s)} f(u)
    if f(s') ≤ f(s) then
        return s
```

end if
$s \leftarrow s'$
end while
return s

2.2 Algoritma Simulated Annealing

Simulated Annealing adalah algoritma pencarian lokal stokastik yang meniru proses pendinginan material untuk mencari solusi bernilai objektif tinggi. Berbeda dari Hill Climbing yang hanya menerima perpindahan ke solusi lebih baik, Simulated Annealing juga dapat menerima solusi yang lebih buruk dengan peluang tertentu. Mekanisme ini membantu pencarian keluar dari optimum lokal dan mengeksplorasi bagian lain dari ruang solusi [2, 3].

Pada setiap iterasi, algoritma memilih satu solusi tetangga dan menghitung perubahan nilai objektifnya. Untuk persoalan maksimisasi, perubahan tersebut didefinisikan sebagai $\Delta E = f(s') - f(s)$, dengan s sebagai solusi saat ini dan s' sebagai kandidat tetangga. Kandidat yang meningkatkan atau mempertahankan nilai objektif selalu diterima. Kandidat yang menurunkan nilai objektif dapat diterima dengan peluang yang bergantung pada temperatur T, seperti dirumuskan pada Persamaan 2.1 [3].

$$P(\text{menerima } s') = \begin{cases} 1, & \Delta E \geq 0, \\ \exp\left(\dfrac{\Delta E}{T}\right), & \Delta E < 0. \end{cases} \tag{2.1}$$

Temperatur mengendalikan tingkat eksplorasi algoritma. Pada temperatur tinggi, penurunan nilai objektif masih mungkin diterima sehingga pencarian dapat menjelajahi ruang solusi dengan lebih luas. Ketika temperatur diturunkan, peluang menerima perpindahan yang lebih buruk ikut mengecil dan pencarian semakin berfokus pada perbaikan. Salah satu jadwal pendinginan yang umum digunakan adalah pendinginan geometris, dengan $0 < \alpha < 1$ sebagai faktor pendinginan [2].

$$T_{k+1} = \alpha T_k, \qquad 0 < \alpha < 1. \tag{2.2}$$

Keunggulan Simulated Annealing adalah kemampuannya melewati optimum lokal dengan menerima perpindahan yang sesaat menurunkan nilai objektif, tanpa harus menyimpan banyak kandidat solusi. Namun, kualitas hasil dan waktu pencarian sensitif terhadap temperatur awal, faktor pendinginan, temperatur minimum, serta mekanisme pembentukan tetangga. Jika temperatur turun terlalu cepat, pencarian masih dapat terjebak pada optimum lokal; frekuensi kondisi ini menjadi salah satu metrik evaluasi algoritma. Karena itu, algoritma ini tidak menjamin solusi global terbaik untuk jadwal pendinginan berdurasi terbatas dan hasilnya dapat berbeda antarpercobaan [2, 3].

Simulated Annealing cocok digunakan pada ruang solusi besar dengan banyak optimum lokal, terutama ketika evaluasi solusi cukup murah dan solusi mendekati optimal sudah bermanfaat. Dalam persoalan pemuatan paket, pencarian dimulai dari konfigurasi acak. Pada setiap iterasi, satu tetangga dibentuk melalui operasi pertukaran paket, pemindahan paket, atau rotasi. Algoritma menerima kandidat yang meningkatkan nilai objektif, dan sesekali menerima kandidat yang menurunkannya sesuai peluang pada Persamaan 2.1. Temperatur kemudian dikurangi berdasarkan jadwal pendinginan. Nilai $\exp(\Delta E/T)$ menjadi ukuran peluang penerimaan kandidat yang lebih buruk, sesuai metrik yang diminta spesifikasi untuk dianalisis terhadap iterasi.

Pseudocode berikut menggunakan pendinginan geometris dan berhenti ketika temperatur mencapai nilai minimum. Variabel s_{best} menyimpan konfigurasi bernilai tertinggi yang ditemukan, meskipun proses menerima perpindahan sementara ke konfigurasi yang lebih buruk.

Algorithm 2: Simulated Annealing

Data: Solusi awal s_0, fungsi tetangga N, fungsi objektif f, parameter $T_0 > T_{\min} > 0$ dan $0 < \alpha < 1$
Result: Solusi terbaik yang ditemukan s_{best}

$s \leftarrow s_0$, $s_{\text{best}} \leftarrow s_0$, $T \leftarrow T_0$
while $T > T_{\min}$ **do**
  if $N(s) = \emptyset$ **then**
    return s_{best}
  end if
  Pilih s' secara acak dari $N(s)$
  $\Delta E \leftarrow f(s') - f(s)$
  if $\Delta E \geq 0$ **then**
    $s \leftarrow s'$
  else
    Bangkitkan u secara acak dari $[0, 1]$
    if $u < \exp(\Delta E / T)$ **then**
      $s \leftarrow s'$
    end if
  end if
  if $f(s) > f(s_{\text{best}})$ **then**
    $s_{\text{best}} \leftarrow s$
  end if
  $T \leftarrow \alpha T$
end while
return s_{best}

2.3 Genetic Algorithm

Genetic Algorithm adalah metode pencarian berbasis populasi yang memperbarui kumpulan kandidat solusi melalui mekanisme yang terinspirasi oleh seleksi alam. Setiap kandidat disebut individu, sedangkan nilai objektifnya berperan sebagai ukuran fitness. Pada tiap generasi, algoritma memilih individu sebagai induk, menggabungkan bagian solusi melalui crossover, lalu menerapkan mutasi secara acak untuk membentuk populasi baru [2, 4].

Karakteristik Genetic Algorithm adalah pencarian atas banyak kandidat secara bersamaan dan penggunaan operator stokastik untuk menghasilkan generasi berikutnya. Seleksi roulette memberi peluang lebih besar kepada individu dengan fitness lebih tinggi. Jika fitness individu ke-i adalah f_i dan bobot seleksinya w_i, peluang memilih individu tersebut dirumuskan sebagai

$$P(i) = \frac{w_i}{\sum_{j=1}^{n} w_j}. \tag{2.3}$$

Untuk persoalan pemuatan paket dengan nilai objektif nonnegatif, bobot seleksi dapat menggunakan nilai objektif. Jika seluruh bobot bernilai nol, seleksi dilakukan secara seragam. Crossover satu titik menggabungkan gen dari dua induk, sementara mutasi mengubah satu gen dengan peluang tertentu. Kedua operator harus menghasilkan individu yang memenuhi batasan pemuatan paket [4].

Keunggulan Genetic Algorithm adalah kemampuannya mengeksplorasi beberapa wilayah ruang solusi secara paralel dan mengurangi ketergantungan pencarian pada satu konfigurasi awal. Keragaman populasi dapat membantu pencarian menghindari optimum lokal. Namun, hasil dan waktu pencarian bergantung pada ukuran populasi, jumlah generasi, operator, serta parameter seperti laju mutasi. Algoritma ini tidak menjamin solusi global terbaik dalam jumlah generasi terbatas, dan setiap generasi memerlukan evaluasi banyak kandidat [2, 4].

Genetic Algorithm sesuai untuk ruang solusi besar dengan representasi kandidat yang dapat digabungkan dan dimutasi, khususnya ketika solusi layak dapat dipulihkan setelah operator diterapkan. Pada persoalan pemuatan paket, satu individu merepresentasikan penempatan seluruh paket. Populasi awal memuat konfigurasi awal yang valid dan konfigurasi valid lain yang dibentuk secara acak. Di setiap generasi, individu dievaluasi dengan fungsi objektif, induk dipilih menggunakan roulette, kemudian crossover satu titik dan mutasi dilakukan untuk membentuk populasi berikutnya. Individu terbaik disimpan terpisah sepanjang pencarian agar solusi terbaik tetap tersedia meskipun tidak diwariskan ke generasi baru.

Pseudocode berikut menggunakan seleksi roulette, crossover satu titik dengan perbaikan kelayakan, dan mutasi

satu gen dengan peluang μ. Pencarian berhenti setelah jumlah generasi yang ditentukan dan mengembalikan individu terbaik yang pernah dievaluasi.

Algorithm 3: Genetic Algorithm

Data: Solusi awal valid s_0, fungsi objektif f, ukuran populasi n, jumlah generasi G, laju mutasi μ
Result: Individu terbaik yang ditemukan s_{best}

```
P ← populasi valid berukuran n yang memuat s_0
s_best ← s_0, f_best ← −∞
for g ← 0 sampai G do
    Evaluasi f(x) untuk setiap x ∈ P
    x_best ← arg max_{x∈P} f(x)
    if f(x_best) > f_best then
        s_best ← x_best, f_best ← f(x_best)
    end if
    if g < G then
        P_baru ← ∅
        while |P_baru| < n do
            Pilih induk p_1 dan p_2 dari P dengan seleksi roulette
            c ← crossover satu titik (p_1, p_2) dan perbaiki agar valid
            Dengan peluang μ, mutasikan satu gen pada c dan pertahankan validitasnya
            Tambahkan c ke P_baru
        end while
        P ← P_baru
    end if
end for
return s_best
```

Bab 3

Implementasi Algoritma

3.1 Representasi State

State merepresentasikan satu konfigurasi pemuatan paket pada sebuah truk. Implementasi menyimpan data paket dan truk secara terpisah, lalu menghubungkan setiap paket dengan posisinya melalui pemetaan dari ID paket ke objek penempatan. Nilai penempatan `None` menandakan paket berada di luar truk, sehingga state tetap mencatat seluruh paket, baik yang dimuat maupun tidak.

Objek `Package` menyimpan ID, dimensi awal, nilai, berat, status mudah pecah, dan ETA. Dimensi disimpan sebagai tiga bilangan bulat, sedangkan orientasi adalah permutasi indeks dimensi yang menentukan urutan ukuran setelah rotasi. Ada enam kemungkinan orientasi, sesuai seluruh permutasi indeks tiga dimensi. Objek `Truck` menyimpan dimensi ruang muat serta kapasitas berat maksimum. Objek `Placement` menggabungkan koordinat sudut paket, dimensi setelah orientasi diterapkan, dan orientasinya. Kode 3.1 memperlihatkan struktur data inti dari `src/model.py`; pemeriksaan validasi dihilangkan dari cuplikan agar fokus pada field representasinya.

Kode 3.1: Struktur inti model state

```python
from dataclasses import dataclass
from itertools import permutations
from typing import Mapping, TypeAlias
Dimensions: TypeAlias = tuple[int, int, int]
Coordinate: TypeAlias = tuple[int, int, int]
Orientation: TypeAlias = tuple[int, int, int]
ORIENTATIONS = tuple(permutations((0, 1, 2)))
@dataclass(frozen=True, slots=True)
class Package:
    id: str
    dimensions: Dimensions
    value: int
    weight: int
    fragile: bool
    eta: int
@dataclass(frozen=True, slots=True)
class Truck:
    dimensions: Dimensions
    max_capacity: int
@dataclass(frozen=True, slots=True)
class Placement:
    coordinate: Coordinate
    oriented_dimensions: Dimensions
    orientation: Orientation = (0, 1, 2)
@dataclass(frozen=True, slots=True)
class State:
    packages: tuple[Package, ...]
    truck: Truck
    placements: Mapping[str, Placement | None]
```

Secara matematis, satu state dituliskan sebagai $S = (P, T, \phi)$, dengan $P = (p_1, \ldots, p_n)$ sebagai kumpulan paket, $T = (\mathbf{L}, C)$ sebagai truk berdimensi $\mathbf{L}$ dan berkapasitas maksimum C, serta ϕ sebagai pemetaan penempatan.

Jika paket p_i memiliki ID id_i dan dimensi $\mathbf{d}_i$, maka representasinya adalah

$$\begin{aligned} I &= \{\mathrm{id}_1, \ldots, \mathrm{id}_n\}, \\ \phi &: I \longrightarrow \{\perp\} \cup \left\{(\mathbf{x}_i, \mathbf{d}_i^{\pi_i}, \pi_i) \;\middle|\; \mathbf{x}_i \in \mathbb{Z}^3_{\geq 0},\ \pi_i \in S_3\right\}, \\ \mathbf{d}_i^{\pi_i} &= (d_{i,\pi_i(1)}, d_{i,\pi_i(2)}, d_{i,\pi_i(3)}). \end{aligned} \tag{3.1}$$

Di sini $\perp$ menyatakan paket tidak dimuat, $\mathbf{x}_i$ adalah koordinat sudut paket di dalam truk, dan S_3 adalah himpunan enam permutasi sumbu yang merepresentasikan orientasi. Domain I memastikan setiap paket memiliki tepat satu entri penempatan. Persamaan 3.1 menjelaskan bentuk representasi; batas ruang, kapasitas, dan batasan fisik lainnya menentukan apakah state tersebut layak.

Pemetaan `placements` harus memiliki tepat satu entri untuk setiap ID paket. Konstruktor `State` memeriksa keunikan ID, kelengkapan kunci pemetaan, dan kecocokan dimensi hasil orientasi dengan dimensi paket. Fungsi objektif state menjumlahkan nilai paket yang memiliki penempatan, sehingga paket di luar truk tidak berkontribusi terhadap nilai solusi.

Contoh berikut menunjukkan sebuah paket di dalam truk dan satu paket yang tidak dimuat. Koordinat menggunakan urutan sumbu x, y, dan z, sedangkan koordinat $(0, 0, 0)$ menandakan sudut bawah paket berada di titik asal ruang muat.

Kode 3.2: Contoh pembentukan state pemuatan

```
from src.model import Package, Placement, State, Truck

package = Package("P001", (3, 4, 1), 60, 5, False, 3)
outside_package = Package("P002", (1, 1, 1), 10, 2, False, 1)
truck = Truck((10, 20, 5), 500)

state = State(
    packages=(package, outside_package),
    truck=truck,
    placements={
        "P001": Placement((0, 0, 0), (3, 4, 1)),
        "P002": None,
    },
)
```

3.2 *Constraint* Permasalahan

State layak apabila semua paket yang ditempatkan berada sepenuhnya di dalam truk, tidak saling bertumpang tindih, berat totalnya tidak melampaui kapasitas, dan tidak ada paket yang melayang atau ditopang paket rapuh. Misalkan $I^+ = \{i \in I \mid \phi(i) \neq \perp\}$ adalah ID paket yang dimuat. Untuk setiap $i \in I^+$, tuliskan penempatan sebagai $\phi(i) = (\mathbf{x}_i, \mathbf{d}_i, \pi_i)$, dengan $\mathbf{x}_i = (x_i, y_i, z_i)$ dan $\mathbf{d}_i = (w_i, d_i, h_i)$ sebagai dimensi setelah rotasi. Kumpulan syarat kelayakannya adalah

$$\mathbf{0} \leq \mathbf{x}_i, \qquad \mathbf{x}_i + \mathbf{d}_i \leq \mathbf{L} \qquad \forall i \in I^+, \tag{3.2}$$

$$\exists k \in \{x, y, z\} : \quad x_{i,k} + d_{i,k} \leq x_{j,k} \ \vee \ x_{j,k} + d_{j,k} \leq x_{i,k} \qquad \forall i, j \in I^+,\ i \neq j, \tag{3.3}$$

$$\sum_{i \in I^+} w_i^{\text{paket}} \leq C. \tag{3.4}$$

$$\begin{aligned} z_i > 0 \Rightarrow \exists j \in I^+,\ j \neq i : \\ z_j + h_j = z_i, \quad \neg\,\mathrm{rapuh}(j), \\ \mathrm{overlap}_{xy}(i, j), \quad \forall i \in I^+. \end{aligned} \tag{3.5}$$

Pada Persamaan 3.3, $d_{i,k}$ menyatakan ukuran paket i pada sumbu k, sedangkan w_i^{paket} pada Persamaan 3.4 menyatakan berat paket agar tidak tertukar dengan lebar pada sumbu x. Predikat $\mathrm{overlap}_{xy}$ bernilai benar ketika proyeksi alas dua paket beririsan dengan luas positif. Implementasi memeriksa dukungan paket melalui

irisan alas tersebut dan kecocokan tinggi bagian atas paket penopang dengan dasar paket yang ditopang. Paket di lantai truk ($z_i = 0$) tidak memerlukan paket penopang. Kode 3.3 menunjukkan pemeriksaan yang dipakai oleh `validate_state`; fungsi bantu irisan satu dimensi dan alas juga disertakan.

Kode 3.3: Validasi batas ruang, tabrakan, kapasitas, dan dukungan

```
from itertools import combinations

from src.model import Package, Placement, State

def _overlap_1d(start_a, size_a, start_b, size_b):
    return start_a < start_b + size_b and start_b < start_a + size_a

def _overlap_xy(a, b):
    return (
        _overlap_1d(a.coordinate[0], a.oriented_dimensions[0],
                    b.coordinate[0], b.oriented_dimensions[0])
        and _overlap_1d(a.coordinate[1], a.oriented_dimensions[1],
                        b.coordinate[1], b.oriented_dimensions[1])
    )

def _overlap_3d(a, b):
    return _overlap_xy(a, b) and _overlap_1d(
        a.coordinate[2], a.oriented_dimensions[2],
        b.coordinate[2], b.oriented_dimensions[2]
    )

def validate_state(state):
    errors = []
    placed = [
        (package, placement)
        for package in state.packages
        if (placement := state.placements[package.id]) is not None
    ]

    for package, placement in placed:
        if any(
            position + size > limit
            for position, size, limit in zip(
                placement.coordinate, placement.oriented_dimensions,
                state.truck.dimensions
            )
        ):
            errors.append(f"package {package.id!r} is outside the truck")

    for (package_a, placement_a), (package_b, placement_b) in combinations(placed, 2):
        if _overlap_3d(placement_a, placement_b):
            errors.append(f"packages {package_a.id!r} and {package_b.id!r} overlap")

    total_weight = sum(package.weight for package, _ in placed)
    if total_weight > state.truck.max_capacity:
        errors.append("truck capacity is exceeded")

    for package, placement in placed:
        if placement.coordinate[2] == 0:
            continue
        supported = any(
            not support_package.fragile
            and support_placement.coordinate[2]
                + support_placement.oriented_dimensions[2]
                == placement.coordinate[2]
            and _overlap_xy(placement, support_placement)
            for support_package, support_placement in placed
            if support_package.id != package.id
        )
        if not supported:
            errors.append(f"package {package.id!r} is unsupported")

    return errors
```

3.3 *Successor Function*

Fungsi suksesor membangkitkan kandidat dengan salah satu dari tiga operasi yang diizinkan: memindahkan satu paket, merotasi satu paket 90 derajat terhadap salah satu sumbu, atau menukar penempatan dua paket. Misalkan $\mathcal{O}(S)$ adalah himpunan operasi yang tersedia pada state S, dengan operasi swap disertakan hanya jika terdapat setidaknya dua paket. Himpunan suksesor layaknya didefinisikan sebagai

$$N(S) = \{S' \mid S' = o(S),\ o \in \mathcal{O}(S),\ S' \neq S,\ \text{validate}(S') = \emptyset\}. \tag{3.6}$$

Operasi move memilih paket secara acak, mempertahankan orientasinya jika paket sudah dimuat, lalu memilih koordinat baru yang masih muat di truk. Jika paket belum dimuat, orientasinya dipilih secara acak. Operasi rotate memilih paket yang sedang dimuat dan menukar dua sumbu orientasinya tanpa mengubah koordinat. Operasi swap memilih dua paket dan menukar status atau posisi pemuatannya. Implementasi memilih operasi secara acak, mencoba paling banyak 100 kandidat, lalu mengembalikan kandidat pertama yang berbeda dari state asal dan lolos validasi. Jika tidak ada paket atau tidak ditemukan kandidat layak dalam batas percobaan, fungsi mengembalikan state asal. Cuplikan 3.4 memperlihatkan pemilihan operasi dan penyaringan tersebut.

Kode 3.4: Pemilihan kandidat suksesor yang valid

```
from random import Random

def random_neighbor(state, rng=None, max_attempts=100):
    if max_attempts < 1 or not state.packages:
        return state
    rng = rng or Random()
    operations = [_move_candidate, _rotate_candidate]
    if len(state.packages) > 1:
        operations.append(_swap_candidate)

    for _ in range(max_attempts):
        candidate = rng.choice(operations)(state, rng)
        if candidate is not state and _valid(candidate):
            return candidate
    return state
```

3.4 *Objective Function*

3.5 Formulasi *Initial State*

3.6 Implementasi Hill Climbing

3.7 Implementasi Simullated Annealing

3.8 Implementasi Genetic Algorithm

Bab 4

Hasil Eksperimen dan Analisis

4.1 Metodologi Eksperimen

4.2 Hasil Eksperimen dan Analisis tiap Algoritma

4.2.1 Hill Climbing

4.2.2 Simulated Annealing

4.2.3 Genetic Algorithm

4.3 Analisis Keseluruhan

Bab 5

Kesimpulan dan Saran

5.1 Kesimpulan

5.2 Saran

Bibliografi

[1] D. L. Poole and A. K. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*, 2nd ed. Cambridge University Press, 2017. [Online]. Available: https://artint.info/2e/html2e/ArtInt2e.Ch4.S7.SS1.html

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[3] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[4] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.